

Session 07 – Integrating React with APIs (Assignment)

1. Objective

Session 07 focused on integrating the React frontend with the FastAPI backend API.

Main objectives:

- Connect React to FastAPI using Axios.
- Create a reusable Axios client.
- Organize API calls into separate modules.
- Implement shared error handling.
- Retrieve product list and product details from the backend API.

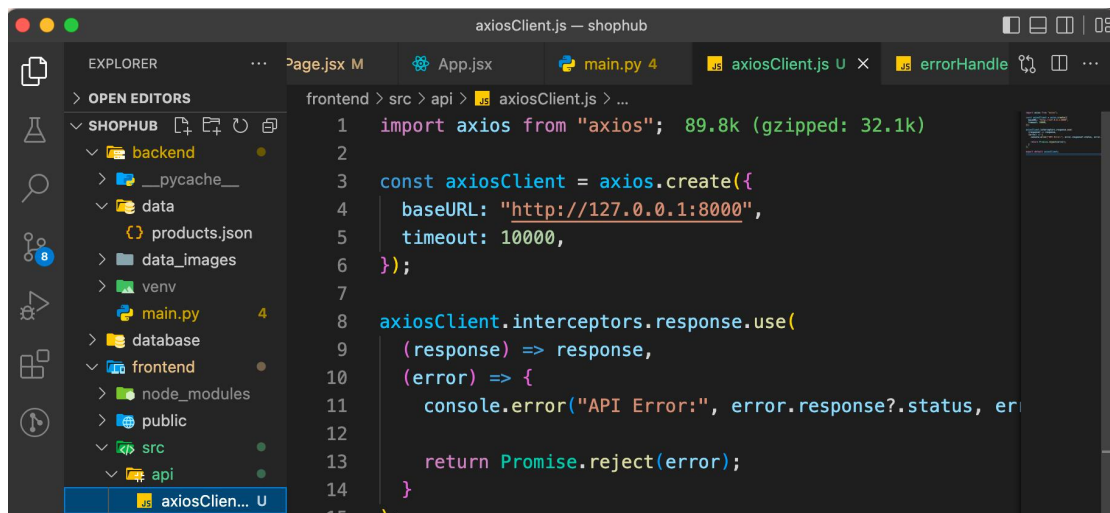
2. Main Concepts

2.1 Axios

Axios is an HTTP client used to communicate with backend APIs.

Features used:

- `axios.create()`
- GET requests
- `async/await`



```
1 import axios from "axios"; 89.8k (gzipped: 32.1k)
2
3 const axiosClient = axios.create({
4   baseUrl: "http://127.0.0.1:8000",
5   timeout: 10000,
6 });
7
8 axiosClient.interceptors.response.use(
9   (response) => response,
10  (error) => {
11    console.error("API Error:", error.response?.status, error);
12
13    return Promise.reject(error);
14  }
15 );
```

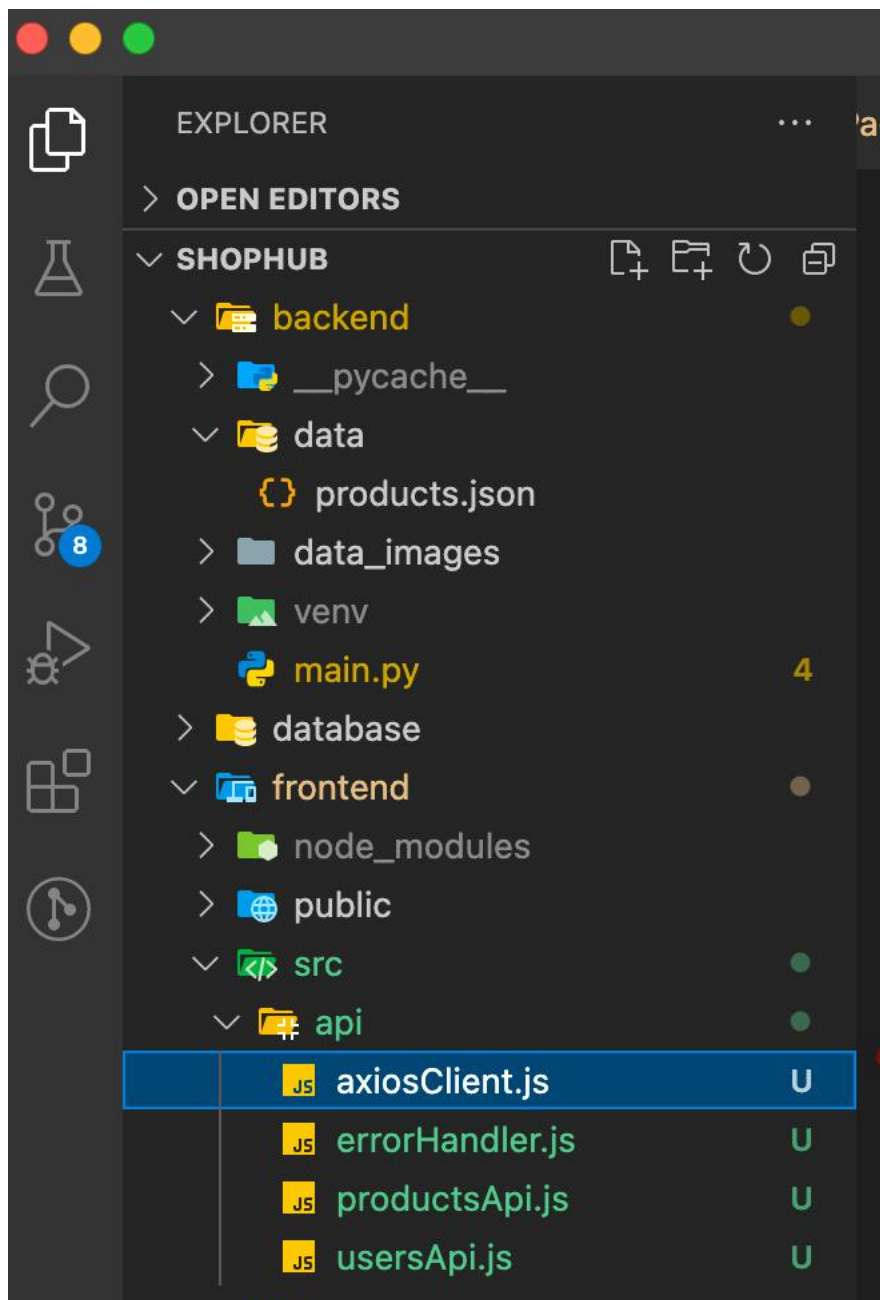
2.2 API Modules

API logic was separated into reusable modules:

- productsApi.js
- usersApi.js

Benefits:

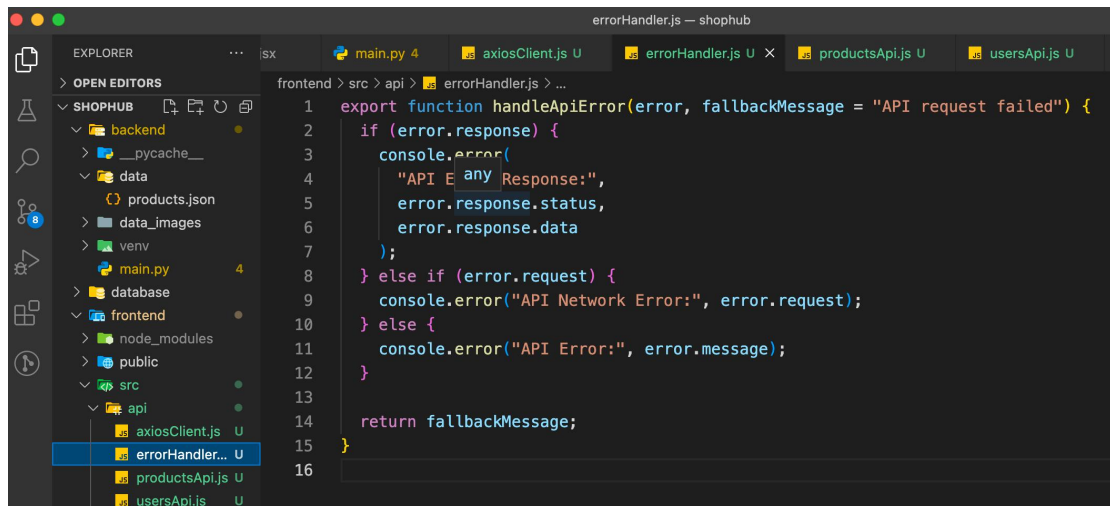
- Cleaner code
- Easier maintenance
- Reusable API functions



2.3 Error Handling

A shared function `handleApiError()` was created to handle:

- Server errors (4xx, 5xx)
- Network errors
- Other unexpected errors



3. Implementation

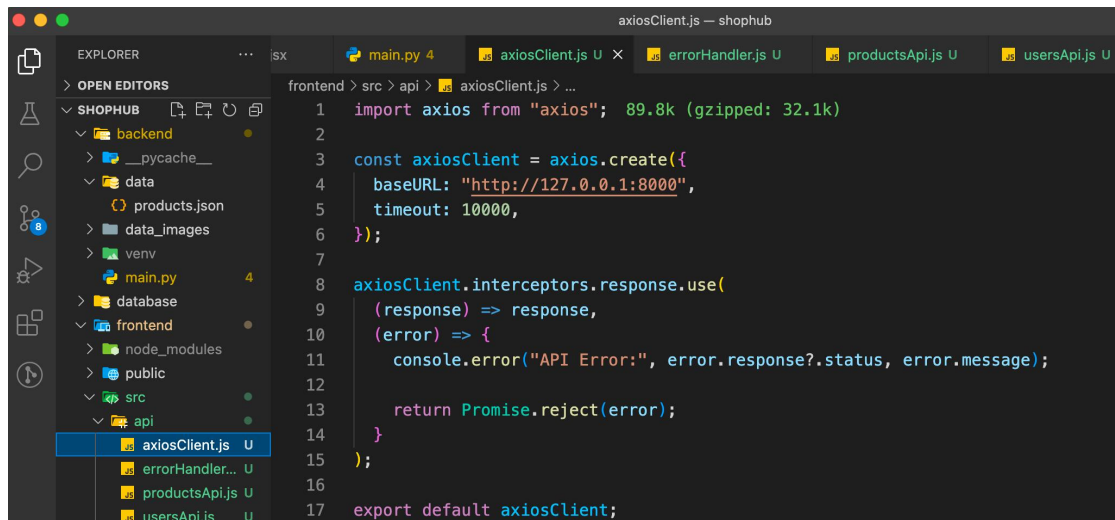
3.1 Axios Client

Created:

`src/api/axiosClient.js`

Configured:

- baseURL
- timeout
- response interceptor



```
1 import axios from "axios"; 89.8k (gzipped: 32.1k)
2
3 const axiosClient = axios.create({
4   baseUrl: "http://127.0.0.1:8000",
5   timeout: 10000,
6 });
7
8 axiosClient.interceptors.response.use(
9   (response) => response,
10  (error) => {
11    console.error("API Error:", error.response?.status, error.message);
12  }
13  return Promise.reject(error);
14 );
15
16
17 export default axiosClient;
```

3.2 Products API

Created:

src/api/productsApi.js

Functions:

- getAll()

- `getById()`

```

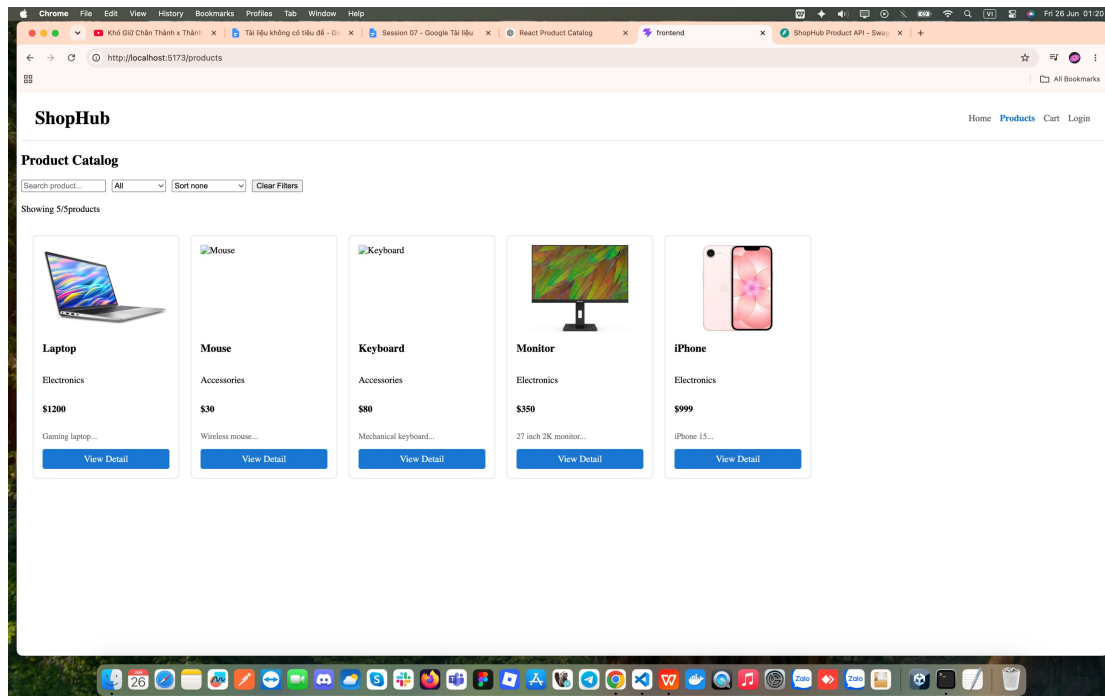
1  import axiosClient from "../axiosClient";
2  import { handleApiError } from "../errorHandler";
3
4  export const productsApi = {
5    async getAll(params = {}) {
6      try {
7        const response = await axiosClient.get("/products", { params });
8        return response.data;
9      } catch (error) {
10       handleApiError(error, "Failed to fetch products");
11       throw error;
12     }
13   },
14   async getById(id) {
15     try {
16       const response = await axiosClient.get(`/products/${id}`);
17       return response.data;
18     } catch (error) {
19       handleApiError(error, "Failed to fetch product detail");
20       throw error;
21     }
22   },
23 };

```

3.4 Product List Integration

Updated ProductPage:

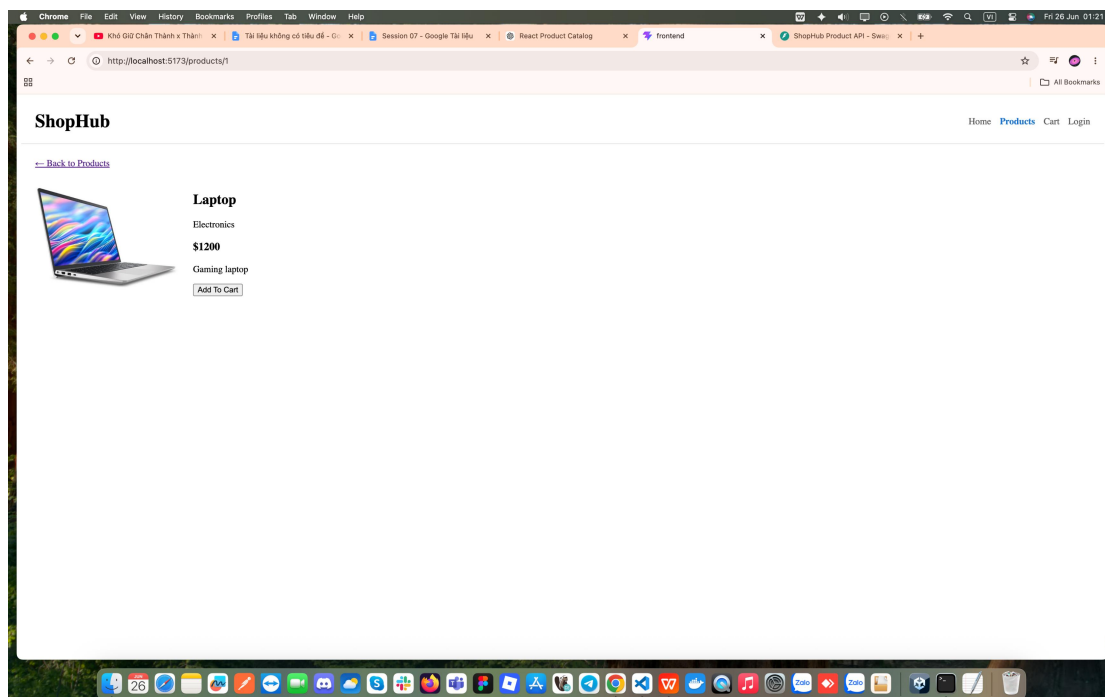
- Uses `productsApi.getAll()`
- Loads data from FastAPI backend
- Displays product list dynamically



3.5 Product Detail Integration

Updated ProductDetailPage:

- Uses `productsApi.getById(id)`
- Retrieves product detail from backend
- Displays product information and image

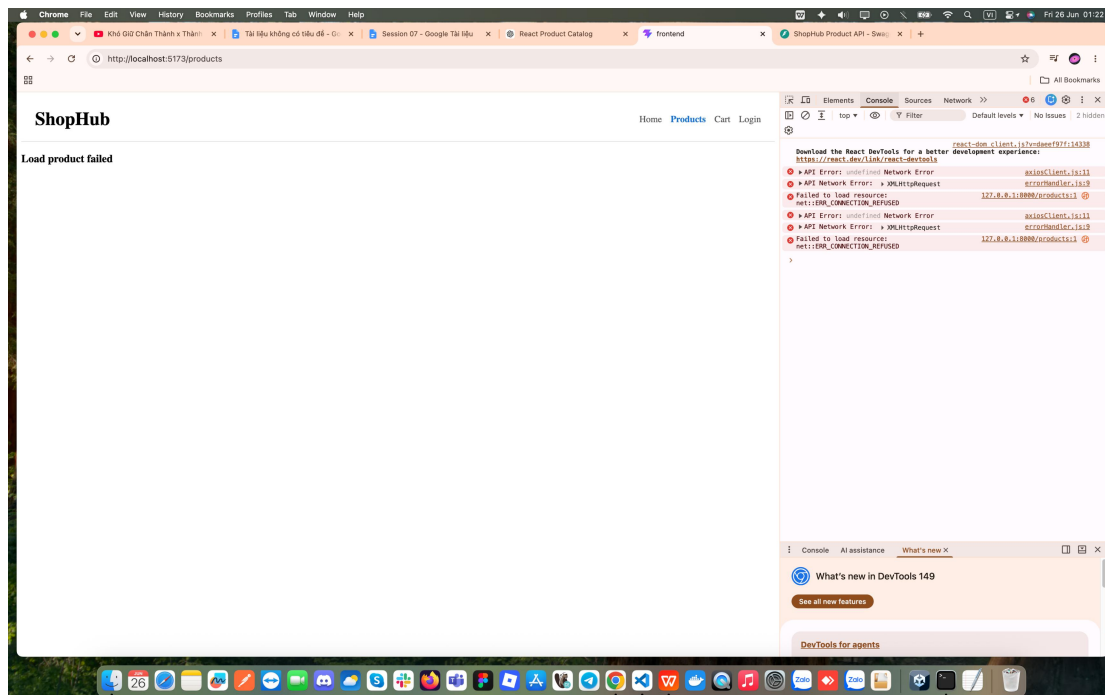


3.6 Error Handling Test

Tested by stopping the backend server.

Result:

- Error logged in browser console.
- Friendly message displayed on the UI.



4. Results

After completing Session 07:

- React successfully communicates with FastAPI.
- Product data is loaded from backend APIs.
- Product detail pages use dynamic API requests.
- API logic is centralized through reusable modules.
- Error handling is standardized across the application.
- Frontend architecture is more scalable and maintainable.